Inside LC0's Mind: A Visual Guide

An Illustrated Step-by-Step Exposition of Neural MCTS

Chess Speak Out Loud Project

August 9, 2026

Contents

Chapter 1

Part 0: Foundations — The Mathematician's Toolset

Before analyzing search tree iterations or neural network evaluations, we must establish the core statistical and computational tools that govern Monte-Carlo Tree Search (MCTS). This chapter provides the rigorous mathematical foundations for every quantity and equation appearing in this guide.

1.1 How to Read These Pictures

Every search-tree diagram in this document is drawn using a fixed visual grammar. The meaning of every border, color, arrow, and bar is defined here once and remains constant throughout the guide.

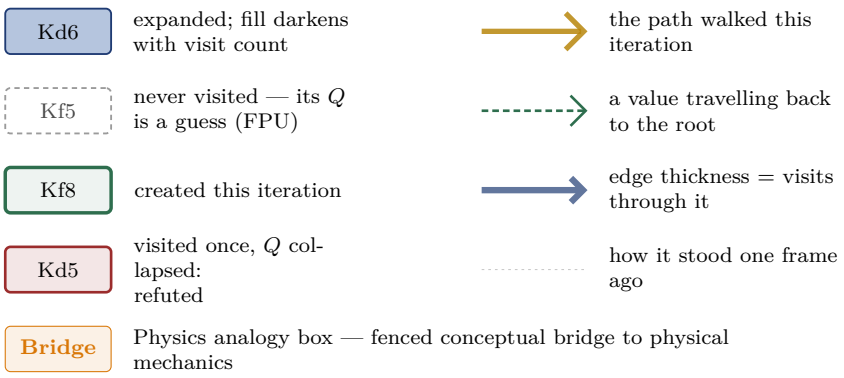


Figure 1.1: FIG-0.1: The visual grammar legend. Node borders, fill saturations, path highlights, edge thicknesses, and physics bridge boxes maintain byte-identical definitions across all tree frames.

Key idea

Solid blue borders represent expanded nodes evaluated by the network; dashed grey borders represent unvisited nodes whose Q value is an FPU (First Play Urgency) guess. The thick green border marks a node created on the current iteration, while red fill indicates a refuted move where Q collapsed.

1.2 Notation and Conventions

To navigate tree search equations unambiguously, we maintain strict distinctions between absolute state properties and side-to-move relative evaluations.

Core Symbol & Notation Reference Table

Symbol	Name / Meaning	Type / Range	Whose Frame / Point of View
s	Position state (tree node)	Discrete FEN	Absolute
a	Candidate move (tree edge)	Action move	Absolute
$V(s)$	Single-look network value	$\in [-1, +1]$	**Side-to-move relative** (Negamax)
w, d, l	Win, Draw, Loss probabilities	$\in [0, 1]$, $w + d + l = 1$	Absolute
$P(a \mid s)$	Policy prior probability	$\in [0, 1]$, $\sum_a P = 1$	Absolute
$N(s)$	Total node visit count	Integer ≥ 0	Absolute
$n_a = N(s, a)$	Move visit count	Integer ≥ 0	Absolute
$W(s, a)$	Accumulated backed-up value	$\in [-\infty, +\infty]$	**Side-to-move relative**
$Q(s, a)$	Running average value W/n_a	$\in [-1, +1]$	**Side-to-move relative**
$U(s, a)$	Curiosity / exploration bonus	≥ 0	**Side-to-move relative**
$S(a)$	Total selection score $Q + U$	$\in [-\infty, +\infty]$	**Side-to-move relative**
$c_{\text{puct}}(N)$	CPUCT exploration parameter	> 0	Absolute
Q_{FPU}	First Play Urgency unvisited floor	$\in [-1, +1]$	**Side-to-move relative**
$\sum_{\text{vis}} P$	Visited policy prior mass	$\in [0, 1]$	Absolute

Figure 1.2: FIG-0.2: Core Symbol & Notation Reference Table. Every symbol used in the UCB1, UCT, and PUCT equations, detailing type, domain range, and frame-of-reference (side-to-move relative vs absolute).

Established mechanism: Negamax Frame of Reference

All scalar evaluation quantities ($V, W, Q, U, S, Q_{\text{FPU}}$) are always expressed **from the perspective of the player to move at that specific node**. If White is to move at node s, $V(s) = +0.97$ means White is winning; if Black is to move at node s', $V(s') = +0.95$ means Black is winning. Absolute quantities (N, n_a, P, w, d, l) do not change sign.

1.3 What the Numbers Mean: Score, Expectation, WDL

Traditional engines output evaluations in **centipawns** (cp), where +100 cp represents a one-pawn advantage. Leela Chess Zero (LC0) operates natively on probability distributions over game outcomes: Win (w), Draw (d), and Loss (l), subject to the simplex constraint $w + d + l = 1$.

1.3.1 Expected Score and Net Value

The observable outcome of a chess game yields 1.0 for a win, 0.5 for a draw, and 0.0 for a loss. The expected score $\mathbb{E}[\text{score}]$ of a position is:

$$\boxed{\mathbb{E}[\text{score}] \;=\; w \;+\; \frac{1}{2}d} \tag{1.1}$$

For internal search arithmetic, LC0 maps the expected score onto the symmetric interval $[-1, +1]$ via net value V:

$$\boxed{V \;=\; w \;-\; l \;\in\; [-1, +1]} \tag{1.2}$$

where $V = +1$ represents a guaranteed win, $V = 0$ represents a balanced draw, and $V = -1$ represents a guaranteed loss. Draw probability is recovered via $d = 1 - w - l$.

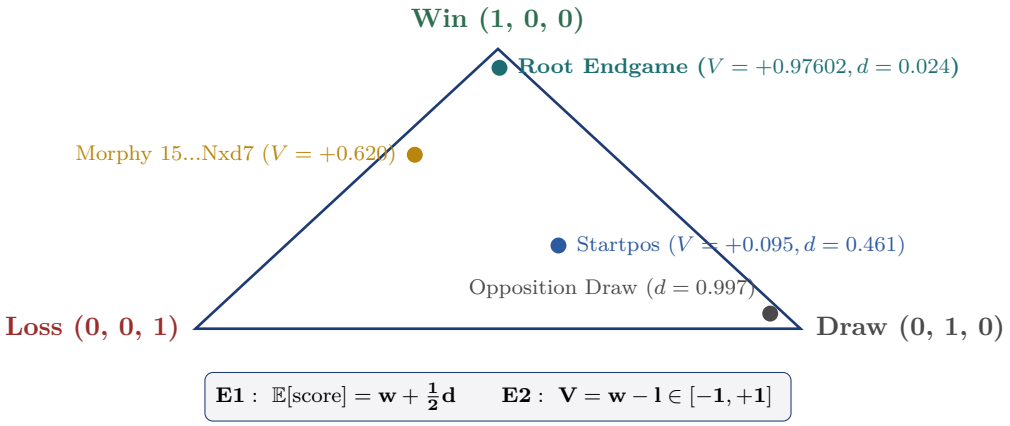


Figure 1.3: FIG-0.3: Expected score and the WDL Simplex. The barycentric simplex $w+d+l = 1$. Measured engine data points are plotted for the initial position, an opposition draw, the root endgame, and Morphy's Opera game position.

1.3.2 Centipawn Conversion

When communicating with UCI chess GUIs, centipawn evaluations are derived from expected score using a logistic mapping:

$$\boxed{\mathbb{E}[\text{score}] \;=\; \frac{1}{1 \;+\; 10^{-\text{cp}/400}}} \tag{1.3}$$

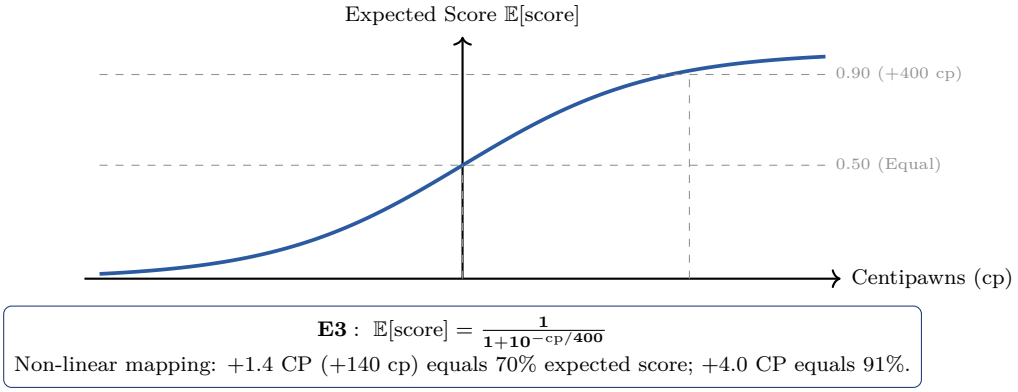


Figure 1.4: FIG-0.4: Logistic Centipawn ↔ Probability conversion curve. Demonstrating the non-linear saturation of centipawns: $+140\,\text{cp} \approx 70\%$ expected score, while $+400\,\text{cp} \approx 91\%$.

1.4 Why a Move's Value Must Be Sampled

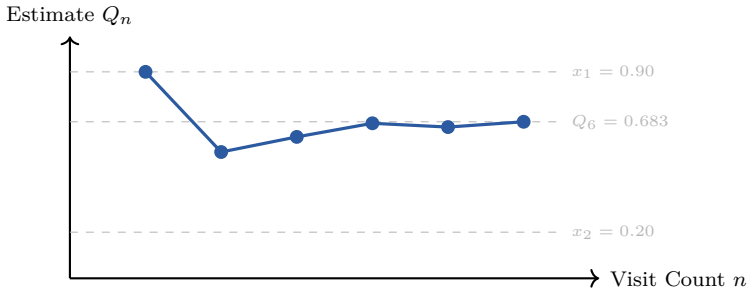
A move's true value μ is the outcome under minimax play from that move — a quantity far beyond exhaustive calculation. In MCTS, we estimate μ by examining continuations. Each visit to move a yields a single-look evaluation $x_i \in [-1, +1]$.

1.4.1 The Incremental Sample Mean

After n visits returning samples $x_1, x_2, \ldots, x_n$, the estimated value Q_n is the sample mean:

$$\boxed{Q_n \;=\; \frac{W_n}{n} \;=\; Q_{n-1} \;+\; \frac{1}{n}\left(x_n - Q_{n-1}\right)} \tag{1.4}$$

where $W_n = \sum_{i=1}^{n} x_i$ is the accumulated backed-up value.



E4 : $\mathbf{Q_n = Q_{n-1} + \frac{1}{n}\left(x_n - Q_{n-1}\right) = \frac{W_n}{n}}$
Early samples step by $\frac{1}{2}$ (0.35 move); sample 6 steps by $\frac{1}{6}$ (0.023 move).

Figure 1.5: FIG-0.5: Incremental sample mean convergence. Demonstrating how Q_n settles over 6 visits. Early samples cause large shifts ($\frac{1}{2}$ step size), while later samples damp fluctuations ($\frac{1}{6}$ step size).

If you know this from physics: Law of Large Numbers and Non-Uniform Sampling

By the Law of Large Numbers (LLN), the sample mean $Q_n = W_n/n$ converges to the expected outcome under repeated evaluations. Classical Monte Carlo estimates expectations by sampling uniformly across state space. MCTS, by contrast, relies on **deliberately non-uniform sampling**: it focuses computational budget heavily on high-prior and high-value branches, turning an intractable sum over the full game tree into an efficient, convergent estimator.

1.5 How Wrong Is a Sample Mean?

An engine cannot rank moves solely by Q_n; it must quantify uncertainty to decide where to allocate further visits.

1.5.1 The $1/\sqrt{n}$ Standard Error Law

If samples are independent draws with standard deviation σ, the standard error of Q_n is:

$$\boxed{\mathrm{SE}(Q_n) \;=\; \frac{\sigma}{\sqrt{n}}} \tag{1.5}$$

1.5.2 Concentration Bounds and Confidence Radius

Using Hoeffding's inequality for bounded random variables in interval width $R = 2$, the probability that the true mean μ exceeds Q_n by more than ϵ satisfies:

$$\Pr(\mu \geq Q_n + \epsilon) \leq \exp\left(-\frac{2n\epsilon^2}{R^2}\right)$$

Inverting this for failure probability δ yields the confidence radius ϵ:

$$\boxed{\epsilon \;=\; R\sqrt{\frac{\ln(1/\delta)}{2n}}} \tag{1.6}$$

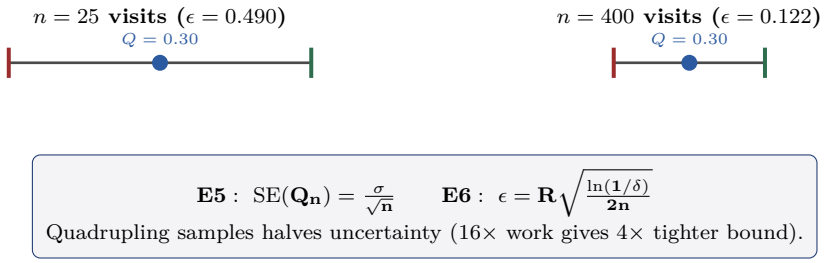


Figure 1.6: FIG-0.6: Standard error $1/\sqrt{n}$ law and confidence radius. Driving uncertainty down from $\epsilon = 0.490$ ($n = 25$) to $\epsilon = 0.122$ ($n = 400$) requires a 16× increase in visits.

If you know this from physics: Diffusion and Random Walk Scaling

In Brownian motion, the root-mean-square displacement scales as $\Delta x_{\text{rms}} \propto \sqrt{t}$. The inverse relationship appears in statistical sampling: the error radius shrinks as $1/\sqrt{n}$. Halving spatial uncertainty in a diffusion process requires 4× the elapsed time; halving value uncertainty in MCTS requires 4× the sample size.

1.6 Optimism Under Uncertainty — And Where ln N Comes From

To minimize **regret** (wasted visits on inferior moves), an agent should follow the principle of **Optimism in the Face of Uncertainty**: rank options by their Upper Confidence Bound $Q_i + \epsilon_i$.

1.6.1 Union Bounds and UCB1

Why must a logarithm appear in confidence bounds rather than a power or linear term? The logarithm arises directly from demanding simultaneous confidence over multiple events:

(a) **Single-Event Radius**: For one arm i at visit count n_i, Hoeffding's inequality (Equation (1.6)) bounds single-tail failure probability by $\delta_i = \exp(-2n_i\epsilon_i^2/R^2)$.

(b) **Simultaneous Coverage via Union Bound**: To guarantee that the confidence interval holds *simultaneously* across all K candidate moves at every step up to total visits N, we must protect against $M \approx K \cdot N^p$ potential failure events. By Boole's inequality (the union bound), the total failure probability is bounded by the sum of individual error probabilities:

$$\Pr(\text{any bound fails}) \;\leq\; \sum_{i=1}^{M} \delta_i \;\leq\; M \cdot \delta_i$$

To enforce overall failure probability $\delta_{\text{total}} \leq \alpha$, each individual event's failure tolerance must shrink like $\delta_i \sim \frac{\alpha}{KN^p}$.

(c) **Logarithmic Cheapness**: Inverting for the confidence radius yields:

$$\epsilon_i \;=\; R\sqrt{\frac{\ln(1/\delta_i)}{2n_i}} \;=\; R\sqrt{\frac{\ln K + p\ln N + \ln(1/\alpha)}{2n_i}}$$

Because ϵ depends on δ strictly through $\sqrt{\ln(1/\delta)}$, protecting against $M = 10^6$ simultaneous events widens the interval by only $\sqrt{\ln 10^6 / \ln 10} \approx 2.45\times$ compared to protecting $M = 10$. Simultaneous confidence over a massive event space is remarkably cheap — and *that* is why a logarithm appears.

(d) **Design Exponent**: Setting per-step failure rate $\delta = N^{-4}$ yields the classic UCB1 formula (Auer et al., 2002):

$$\boxed{a^* \;=\; \arg\max_i \left[\, Q_i \;+\; c\sqrt{\frac{\ln N}{n_i}} \,\right]} \tag{1.7}$$

The exponent $p = 4$ is a design choice trading interval width against overall confidence, not a law of physics.

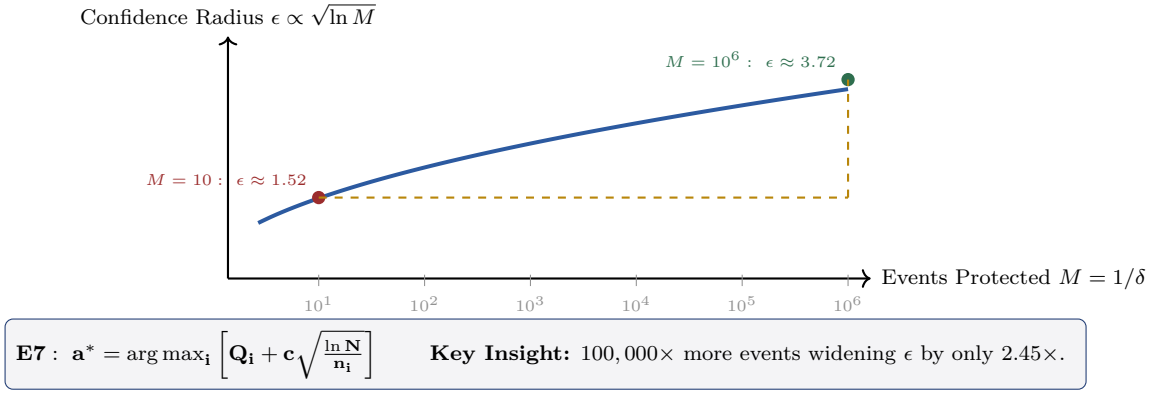


Figure 1.7: FIG-0.7: Simultaneous confidence and UCB1 derivation. The confidence radius $\epsilon \propto \sqrt{\ln M}$ plotted against the number of protected events $M = 1/\delta$. Protecting against 100,000× more events ($M = 10^1 \to 10^6$) widens ϵ by only 2.45×, demonstrating why simultaneous confidence requires only logarithmic width.

1.7 From One Node to a Tree: Nested Bandits and the Sign Flip

Extending single-node bandit selection across an entire game tree produces UCT (Upper Confidence bounds applied to Trees; Kocsis & Szepesvári, 2006).

1.7.1 Negamax Backup and UCT Selection

In a zero-sum game, a position carries opposite net evaluations depending on who is to move: $V_{\text{white}} + V_{\text{black}} = 0$. Because value V is defined relative to the side to move at each node, backpropagation alternates sign at every ply:

$$\boxed{V_{\text{parent}} \;=\; -V_{\text{child}}} \tag{1.8}$$

UCT applies UCB1 locally at every tree node s:

$$\boxed{a^* \;=\; \arg\max_a \left[\, Q(s,a) \;+\; c\sqrt{\frac{\ln N(s)}{n_a}} \,\right]} \tag{1.9}$$

1.8 Why This Is Not Enough for Chess

Standard UCT treats unvisited edges as having infinite bonus ($n_a = 0 \implies U = \infty$), forcing mandatory first visits on all legal moves before any move receives a second look.

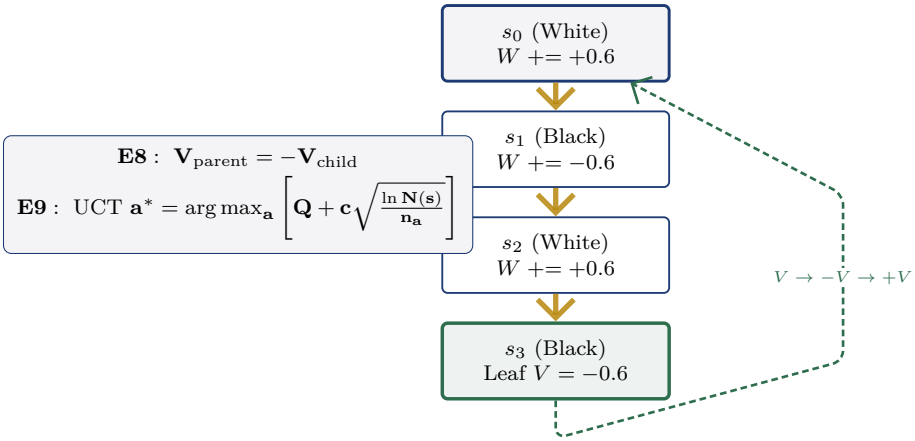


Figure 1.8: FIG-0.8: Nested bandits and Negamax sign flips. Backpropagation path showing $V(s_3) = -0.6$ flipping sign to $+0.6$ at White parent s_2, -0.6 at Black node s_1, and $+0.6$ at White root s_0.

In chess, with an average branching factor of $K \approx 31$, visiting all 46 legal moves at the root consumes 46 visits before a single move can be explored to depth 2. On realistic node budgets, UCT spends almost its entire time probing absurd blunders.

This fundamental limitation necessitates replacing blind statistical exploration with learned prior attention $P(a \mid s)$, leading directly to the **PUCT formula** analyzed in Section 2.2.

1.9 The Equation Register

Every equation introduced across this guide is indexed below for cross-reference.

The Master Equation Register (E1–E12)

#	Equation Formula	Section	Core Role / LC0 Values
E1	$\mathbb{E}[\text{score}] = w + \frac{1}{2}d$	§0.3	Observable expected score
E2	$V = w - l \in [-1, +1]$	§0.3	Net win value (side-to-move)
E3	$\mathbb{E}[\text{score}] = \frac{1}{1+10^{-\text{cp}/400}}$	§0.3	Centipawn to probability logistic map
E4	$Q_n = Q_{n-1} + \frac{1}{n}(x_n - Q_{n-1})$	§0.4	Incremental sample mean update (W/N)
E5	$\text{SE}(Q_n) = \sigma/\sqrt{n}$	§0.5	Standard error $1/\sqrt{n}$ law
E6	$\epsilon = R\sqrt{\ln(1/\delta)/2n}$	§0.5	Hoeffding concentration confidence radius
E7	$a^* = \arg\max_i [Q_i + c\sqrt{\ln N/n_i}]$	§0.6	UCB1 bandit selection rule
E8	$V_{\text{parent}} = -V_{\text{child}}$	§0.7	Negamax two-player value backup
E9	$a^* = \arg\max_a [Q + c\sqrt{\ln N(s)/n_a}]$	§0.7	UCT tree search selection rule
E10	$S(a) = Q(a) + c_{\text{puct}} P(a) \frac{\sqrt{N}}{1+n_a}$	§1.4	PUCT selection formula (LC0 core)
E11	$Q_{\text{FPU}} = Q(\text{parent}) - c_{\text{fpu}}\sqrt{\sum P_{\text{vis}}}$	§1.4	FPU baseline ($c_{\text{fpu}} = 0.33$; FIG-1.10, FIG-1.11a)
E12	$c_{\text{puct}}(N) = c_{\text{base}} + c_{\text{factor}} \ln\left(\frac{N+c_{\text{mod}}}{c_{\text{mod}}}\right)$	§1.4	CPUCT growth ($1.745 + 3.894 \ln \frac{N+38740}{38739}$; FIG-1.10, FIG-1.11d)

Figure 1.9: FIG-0.9: The Master Equation Register (E1–E12). Summary table mapping equation labels, formulas, section introduced, and core role within LC0's search architecture.

Chapter 2

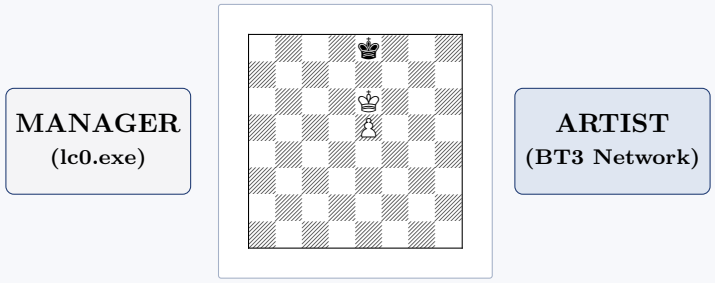
The Two Partners

The search engine operates as a partnership between two distinct components: the **Manager** (`lc0.exe`), which maintains tree bookkeeping and decision selection, and the **Artist** (the BT3 neural network), which evaluates positions in single forward passes.

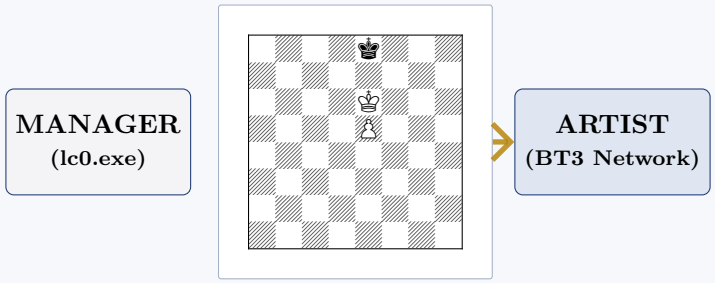
2.1 The Handshake

Established mechanism: The Engine-Network Handshake

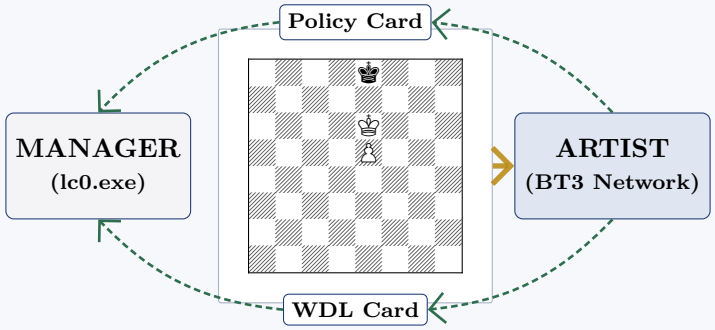
The interaction between manager and neural network proceeds step-by-step:



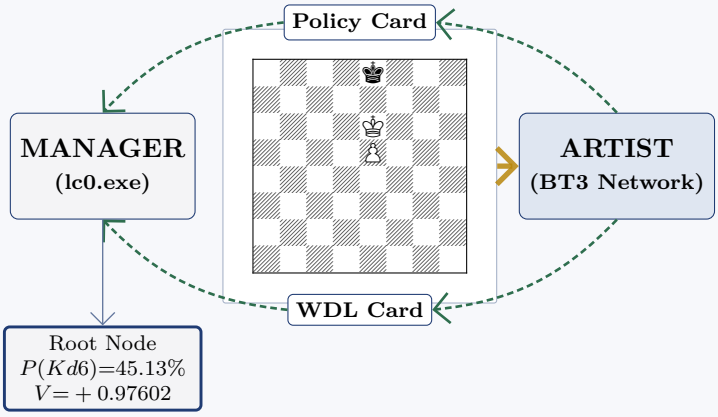
(a) Manager holds board



(b) Board slides to Artist



(c) Artist returns cards



(d) Manager writes node

Figure 2.2: FIG-1.1: The handshake cycle. Manager passes board position to network; network returns policy prior probabilities and single-look WDL value evaluation (Equations (1.1)–(1.2)).

2.2 The Problem the Search Has to Solve

Established mechanism: Three Essential Search Constraints

Before deriving any formula, we set the three requirements (R1–R3) that any MCTS selection rule must satisfy simultaneously:

Three Essential Constraints on Search Selection (R1–R3)
The selection rule must satisfy three competing requirements simultaneously:

R1: Zero Visits Allowed
Most moves must be allowed 0 visits. A 46-move position on a small budget cannot afford 1 visit each.
Prevents: Exhaustive probing.

R2: Reachable Later
Every legal move must remain reachable later. No move may be permanently excluded.
Prevents: Blind spots.

R3: Overrule Prior
With enough visits, measurement must be able to overrule any prior estimate.
Prevents: Prior stubbornness.

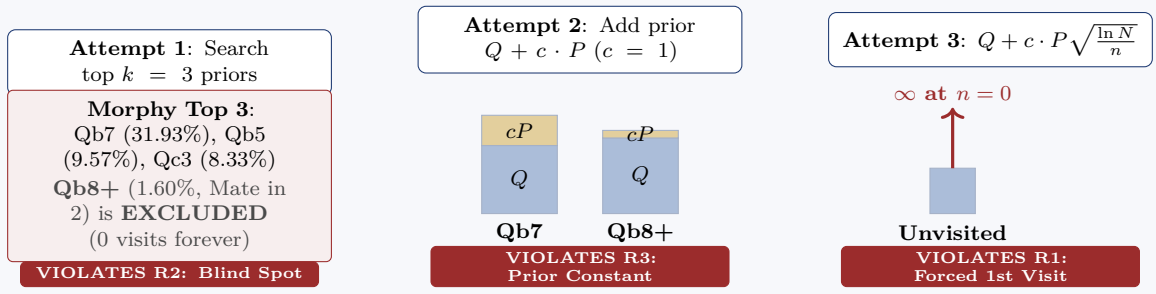
Witness Position: Morphy–Brunswick/Isouard (Paris 1858, 15...Nxd7)
- 46 legal moves. Network prior gives **Qb8+** only $P = \mathbf{1.60}\%$ (ranked 7th, behind Qb7 31.93%, Qb5 9.57%, Qc3 8.33%).
- Yet Stockfish (d30) proves **Qb8+** is a forced mate in 2. Any rule that permanently filters low-prior moves fails here. (`ch06_building_puct.tex:L55`)

Figure 2.3: FIG-1.7: The problem the search has to solve. Citing `ch06_building_puct.tex:L55-L64`. Requirements R1 (zero visits allowed), R2 (reachable later), and R3 (overrule prior) pull against each other. Witness: Morphy position after 15...Nxd7, where forced mate in 2 (**Qb8+**) gets only 1.60% prior attention.

2.3 Building the Selection Rule: Attempts 1–3

Established mechanism: Deriving PUCT: Attempts 1–3

We test candidate selection rules against requirements R1–R3:



(a) 1. Filter top k **(b)** 2. Add prior **(c)** 3. Prior · UCB1

Figure 2.5: FIG-1.8: Selection rule attempts 1–3. Citing `ch06_building_puct.tex:L66-115`. (a) Attempt 1 (top-k filter) fails R2. (b) Attempt 2 ($Q + cP$) fails R3 (prior never decays). (c) Attempt 3 (prior · UCB1) fails R1 (∞ at $n = 0$, modifying Equation (1.7)).

2.4 The Surviving Rule: Attempts 4–5 & Final Formula

Fixing unvisited infinity leads to $\sqrt{N}$ growth and yields the **Predictor + Upper Confidence Bound applied to Trees (PUCT)** selection formula:

$$a^* = \arg\max_a \left[Q(s,a) + c_{\text{puct}}(N) \cdot P(a \mid s) \cdot \frac{\sqrt{N(s)}}{1+n_a} \right] \tag{2.1}$$

Established mechanism: Deriving PUCT: Attempts 4–5 and Final Form

Attempt 4: $Q + c \cdot P\frac{\sqrt{\ln N}}{1+n}$

$N = 100 \rightarrow 10{,}000$ (100× search budget):
$\sqrt{\ln N}$ grows by only **41%** ($\sqrt{2} = 1.41$)
Neglected move needs **10**×
growth to overcome prior deficit

VIOLATES R2: Neglected Move Frozen

(a) 4. Finite at zero

Attempt 5: $Q + c_{\text{puct}} \cdot P\frac{\sqrt{N}}{1+n}$

$N = 100 \rightarrow 10{,}000$ (100× search budget):
$\sqrt{N}$ grows **10-fold** ($10 \rightarrow 100$)
Neglected move gets its turn!

SURVIVOR: Satisfies R1, R2, R3

(b) 5. $\sqrt{N}$ growth

$$\textbf{E10}: \ \mathbf{a}^* = \arg\max_{\mathbf{a}} \left[\underbrace{Q(s,a)}_{\text{what I measured}} + \underbrace{c_{\text{puct}} \cdot P(a \mid s) \cdot \frac{\sqrt{N(s)}}{1+N(s,a)}}_{\mathbf{U(s,a)}\text{: what I haven't checked}} \right]$$

Figure 2.7: FIG-1.9: Selection rule attempts 4–5 and final formula. Citing `ch06_building_puct.tex:L116-149`. (a) Attempt 4 ($\sqrt{\ln N}$ growth) fails R2 in practice because neglected moves stay frozen. (b) Attempt 5 ($\sqrt{N}$ growth) survives R1–R3. Boxed: The complete PUCT selection formula (Equation (2.1)).

2.5 Reading the Finished Formula, Term by Term

Total score $S(a) = Q(a) + U(a)$ combines exploitation $Q(a)$ and exploration $U(a)$. Unvisited nodes inherit an unvisited baseline floor Q_{FPU}:

$$Q_{\text{FPU}} = Q(\text{parent}) - c_{\text{fpu}} \sqrt{\sum_{\text{visited}} P} \tag{2.2}$$

while the CPUCT exploration scaling parameter grows logarithmically with node visit count N:

$$c_{\text{puct}}(N) = c_{\text{base}} + c_{\text{factor}} \ln\left(\frac{N + c_{\text{mod}}}{c_{\text{mod}}}\right) \tag{2.3}$$

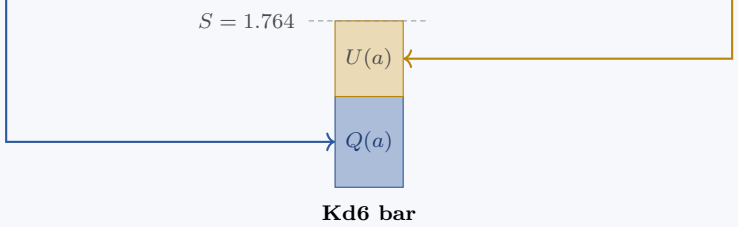
Established mechanism: Math Formula Overview

$$\mathbf{S(a)} = \mathbf{Q(a)} + \mathbf{U(a)} = \mathbf{Q(a)} + \mathbf{c}_{\text{puct}}(\mathbf{N}) \cdot \mathbf{P(a)} \cdot \frac{\sqrt{\max(\mathbf{N}, \mathbf{1})}}{\mathbf{1}+\mathbf{n_a}}$$

Exploitation $Q(a)$ (Solid Bar)
- Visited: Running average of leaf evaluations along branch.
- Unvisited: Inherits Q_{FPU} baseline floor from parent.

Engine Parameters
E12 : $c_{\text{puct}}(N) = 1.745 + 3.894 \cdot \ln\left(\frac{N+38740}{38739}\right)$
E11 : $Q_{\text{FPU}} = Q(\text{parent}) - 0.33 \cdot \sqrt{\sum_{\text{visited}} P}$

Exploration $U(a)$ (Pale Bar)
- Set by policy prior $P(a)$.
- Denominator $(1+n_a)$ shrinks $U(a)$ on visit (halved on 1st visit).



Kd6 bar

Figure 2.8: FIG-1.10: Math formula overview diagram. Total score $S(a) = Q(a) + U(a)$ defined by Equation (2.1). Exploitation $Q(a)$ (solid bar) uses Equation (1.4) for visited moves and Q_{FPU} (Equation (2.2)) for unvisited moves. Exploration $U(a)$ scales with $c_{\text{puct}}(N)$ (Equation (2.3)). Visiting a move increments $(1 + n_a)$, shrinking $U(a)$ (see FIG-2.2).

2.6 Term-by-Term Reasoning

Established mechanism: Term-by-Term Reasoning

The four core design questions behind the formula's terms:

(a) Why Q is an average & starts as a guess

- $Q(s,a)$ is running mean of leaf evaluations (E4).
- Unvisited moves start at $Q_{\text{FPU}} = Q(\text{root}) - 0.33\sqrt{\sum P_{\text{vis}}}$ (E11).
- $Q = 0$ fails: slander in win, optimism in loss.
- $Q = Q(\text{parent})$ fails: unvisited ties with measured. (See FIG-2.B)

(a) Why Q is an average

(b) Why U is built the way it is

- P in numerator: Move with low prior gets small initial claim.
- $\sqrt{N}$ in numerator: Claim of neglected move **grows** as search spends elsewhere (forces visit in FIG-3.1, finds Qb8+ in FIG-4.1).
- $(1 + n_a)$ in denominator: Visiting a move **halves** its claim immediately (halving decides pick in FIG-2.2).

(b) Why U is built this way

(c) Why $S = Q + U$ is a sum & Priority Nature

- Q = how good it looked; U = how much still unknown. Neither can veto the other.
- S is a **priority score** (where to look next), NOT an evaluation or probability.
- **The Trap**: Leading S falls from 1.764 → 1.400 across FIG-2.0→2.8. Q stays flat (+0.976 → +0.968); fall is 100% U decay!

(c) Why $S = Q + U$ is a sum

(d) Why c_{puct} grows with N

- $c_{\text{puct}}(N) = 1.745 + 3.894 \cdot \ln\left(\frac{N+38740}{38739}\right)$ (E12).
- Small N: Nearly constant (1.74520 → 1.74580 over 8 iterations).
- Large N: Reaches $c_{\text{puct}} \approx 14$ at $N = 10^6$, opening search to unexamined moves in deep analysis.

(d) Why c_{puct} grows

Figure 2.10: FIG-1.11: Term-by-term breakdown. Citing `ch06_building_puct.tex:L150-273`. (a) Q is evidence (Equation (1.4)); unvisited moves inherit penalized Q_{FPU} floor (Equation (2.2)). (b) P, $\sqrt{N}$, and $(1 + n_a)$ in Equation (2.1) balance initial claim, search growth, and immediate decay. (c) S is a priority score, not an evaluation; falling S is 100% U decay. (d) $c_{\text{puct}}(N)$ (Equation (2.3)) opens search during deep analysis.

2.7 Anatomy of a Tree Node

A search tree node stores these specific fields **because** the selection rule derived in §1.4 (Equation (2.1)) needs them to evaluate candidate moves.

Established mechanism: Search Tree Node Anatomy

Each node in the MCTS search tree stores both measured network feedback and running statistics:

Kd6
$P(a) = 45.13\%$ $n_a = 4$
$Q(a) = +\mathbf{0.98394}$ $V = +\mathbf{0.97602}$
$U(a) = 0.41691$ $S(a) = 1.40085$

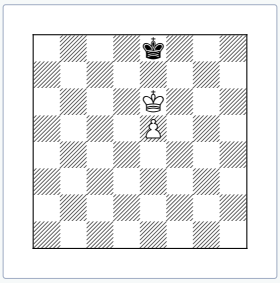
- **Move**: Kd6 candidate move
- **P**: Prior policy probability from network
- **n**: Visit count spent on this branch
- **Q**: Running average evaluation
- **V**: Node's own single-look evaluation
- **U**: Curiosity / exploration bonus
- **S**: Total score $S = Q + U$ (decides pick)

Figure 2.11: FIG-1.2: Node Anatomy. Prior probability $P(a)$, visit count n_a, running average evaluation $Q(a)$ (Equation (1.4)), single-pass value V (Equation (1.2)), curiosity bonus $U(a)$, and selection score $S(a) = Q(a) + U(a)$ (Equation (2.1)).

2.8 The Initial Network Evaluation

Real engine output: Initial Position Assessment

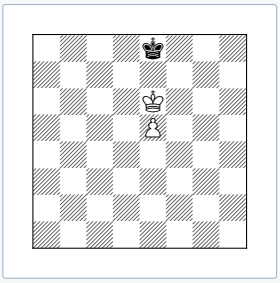
For our subject endgame position (`4k3/8/4K3/4P3/8/8/8/8 w - - 0 1`), one single forward pass produces initial policy priors and win-draw-loss assessment:



Policy Card (Prior Attention)
Kd6: **45.13%** Kf6: **44.23%**
Kf5: 5.38% Kd5: 5.26%

WDL Card (Face Down)
Evaluating position...

(a) Cards Face-Down



Policy Card (Prior Attention)
Kd6: **45.13%** Kf6: **44.23%**
Kf5: 5.38% Kd5: 5.26%

WDL Card (Evaluated)
$V = +\mathbf{0.97602}$ $d = \mathbf{0.024}$

(b) Evaluated Numbers

Figure 2.13: FIG-1.3: Policy and WDL cards for K+P endgame. The policy places 89.36% of attention on winning moves (Kd6 45.13%, Kf6 44.23%), but still leaves 10.64% on drawing moves (Kf5 5.38%, Kd5 5.26%). Initial root value $V = +0.97602$ (Equation (1.2)), $d = 0.024$.

Key idea

The network policy is an informed initial instinct, not an unassailable oracle. 89.36% of its attention went to winning moves, but 10.64% remained on moves that throw away the win. The search exists to verify and refine this instinct.

2.9 One Value Per Position, Not Per Move

Where people go wrong

A common misreading assumes the neural network outputs individual value evaluations for candidate moves:

~~**Kd6**: P=45.13%, V=V_1~~
~~**Kf6**: P=44.23%, V=V_2~~
~~**Kf5**: P=5.38%, V=V_3~~ ~~**Kd5**: P=5.26%, V=V_4~~

The network NEVER outputs a value per move!

(a) The Misreading

One Position Evaluation:
$V(\text{root}) = +0.97602, d = 0.024$
One Policy Distribution: $P = \{45.13\%, 44.23\%, 5.38\%, 5.26\%\}$

One V for the position, four P's for moves. A move gets a value only when expanded into a child position ($V(\text{leaf}) = +0.96766$ in FIG-2.1).

(b) What Actually Comes Back

Figure 2.15: FIG-1.4: One value per position, not one per move. (a) The network does not produce individual V evaluations for candidate moves. (b) A forward pass returns exactly one $V(s)$ (Equation (1.2)) for the current position, alongside policy prior probabilities $P(a \mid s)$ for legal moves. A move receives its own value only when expanded into a child position (see FIG-2.1).

Established mechanism: What the Policy Actually Predicts

The policy prior distribution $P(a \mid s)$ is a fast neural approximation of search preference, not an objective move quality score:

What the Policy Head Models:
*"The policy head predicts **which move a full search by this same engine would end up preferring**. It is a fast approximation of its own slow self. It is not a model of human choice, not a goodness score, and not calibrated to any rating band."* (`ch12_two_heads.tex:L96`)

1. Policy is NOT goodness (Downward)
Kf5 and Kd5 hold **10.64%** of root policy, yet both throw away a won game ($Q = 0.000$, SF d30 0.00). (See FIG-4.3)

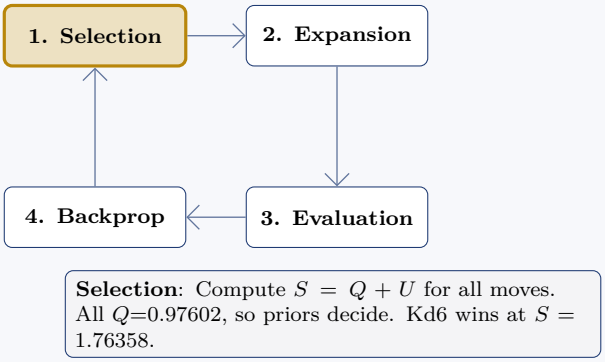
2. Policy is NOT goodness (U
In Morphy position, $P(\text{Qb8+}) =$ on a forced mate in two. Search o policy to find win. (See FIG-5.3)

Figure 2.16: FIG-1.5: What the policy predicts. Citing `ch12_two_heads.tex:L96`. Measured proofs: 10.64% of root policy goes to losing blunders (FIG-4.3), while a forced mate in 2 receives only 1.60% prior attention (FIG-5.3).

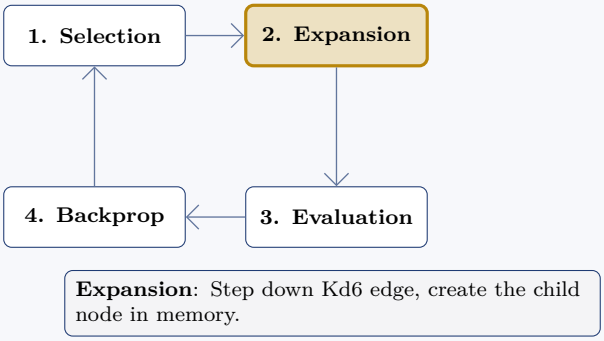
2.10 The Four-Phase Iteration Loop

Established mechanism: The Four-Phase Loop

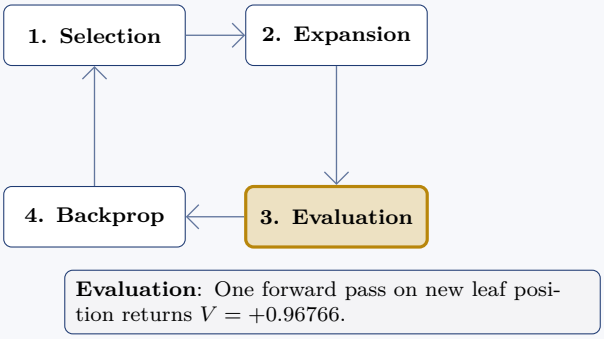
Every single search iteration follows a continuous four-phase loop (Selection → Expansion → Evaluation → Backpropagation):



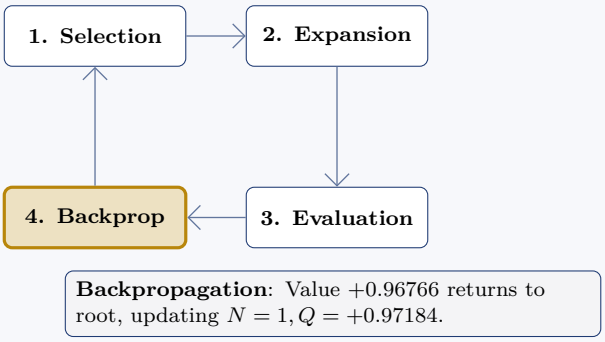
(a) 1. Selection



(b) 2. Expansion



(c) 3. Evaluation



(d) 4. Backpropagation

Figure 2.18: FIG-1.6: The four-phase MCTS iteration cycle. Citing `ch05_machines_to_trees.tex`. Each search iteration executes four sequential steps: Selection (choosing $\max S = Q + U$ via Equation (2.1)), Expansion (creating child node), Evaluation (single forward pass on leaf), and Backpropagation (updating N and Q via Equation (1.4) and sign flip Equation (1.8) along path back to root).

Chapter 3

The Growing Tree

The spine of the guide is an eight-iteration search on our four-legal-move endgame position. Because there are only four legal moves at the root, every single node in the tree can be drawn explicitly at fixed coordinates without schematic shortcuts.

3.1 Selection Rule Recall

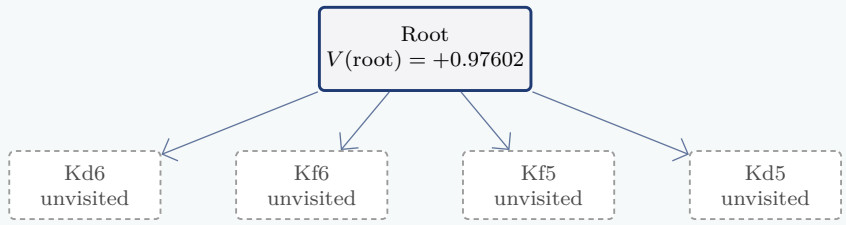
Before tracing the search, we recall the core score calculation derived in §1.4 (Equation (2.1)).

Established mechanism: Selection Rule Recall

Selection score $S(a) = Q(a) + U(a)$ (Equation (2.1)) governs move choice (see **FIG-1.10** in §1.5 for the complete term-by-term derivation and engine parameters). Unvisited moves inherit Q_{FPU} floor (Equation (2.2)) penalized by FPU reduction. The move with the tallest bar ($\max S$) is selected.

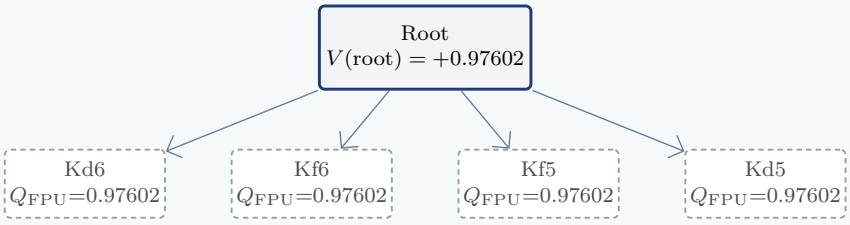
Real engine output: Where the Solid Bar Segment Comes From

The origin of solid bar segments before and after measurement:

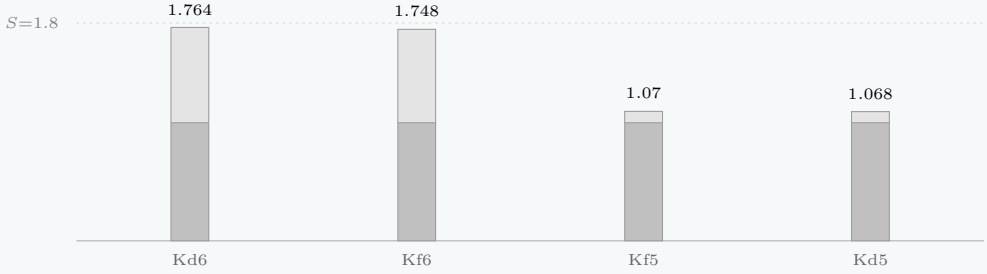


Step 1: V arrives at root. Move bars are empty:

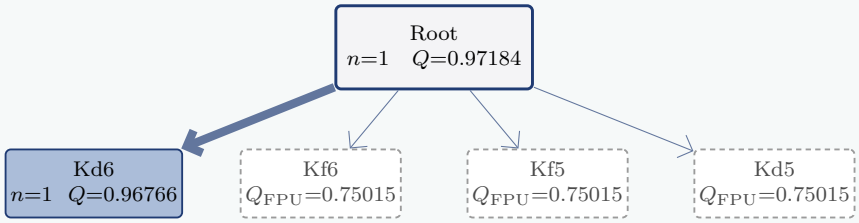
(a) 1. V arrives



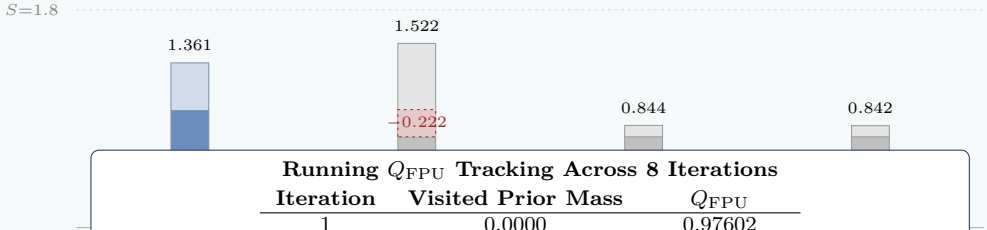
Step 2: V sets Q_{FPU} floor for ALL moves:



(b) 2. V becomes Q_{FPU} floor



Step 3: After Kd6 visit, penalty $0.33\sqrt{0.4513} = 0.22169$ drops unvisited bars:



Running Q_{FPU} Tracking Across 8 Iterations

Iteration	Visited Prior Mass	Q_{FPU}
1	0.0000	0.97602
2	0.4513	0.75015
3	0.8936	0.66460
4–8	0.8936	0.658 – 0.668

The penalty stops growing after iteration 3 because only Kd6 and Kf6 are ever visited, so no new prior mass is added. Q_{FPU} only drifts with root Q (see `ch07:fpudrop`).

(c) 3. FPU penalty appears

3.2 The Eight Search Iterations

We trace the search iteration by iteration. A FIG-2.x frame captures two moments: the **tree** displays the state *after* iteration x completes (new leaf evaluated, visit counts incremented, backpropagated values), while the **S-bar strip** below displays the scores compared *before* iteration x's pick — the numbers that caused the selection. The tallest bar ($S = Q + U$) carries gold highlight and matches the candidate move marked with the gold selection ring in the tree above.

Real engine output: Iteration 0

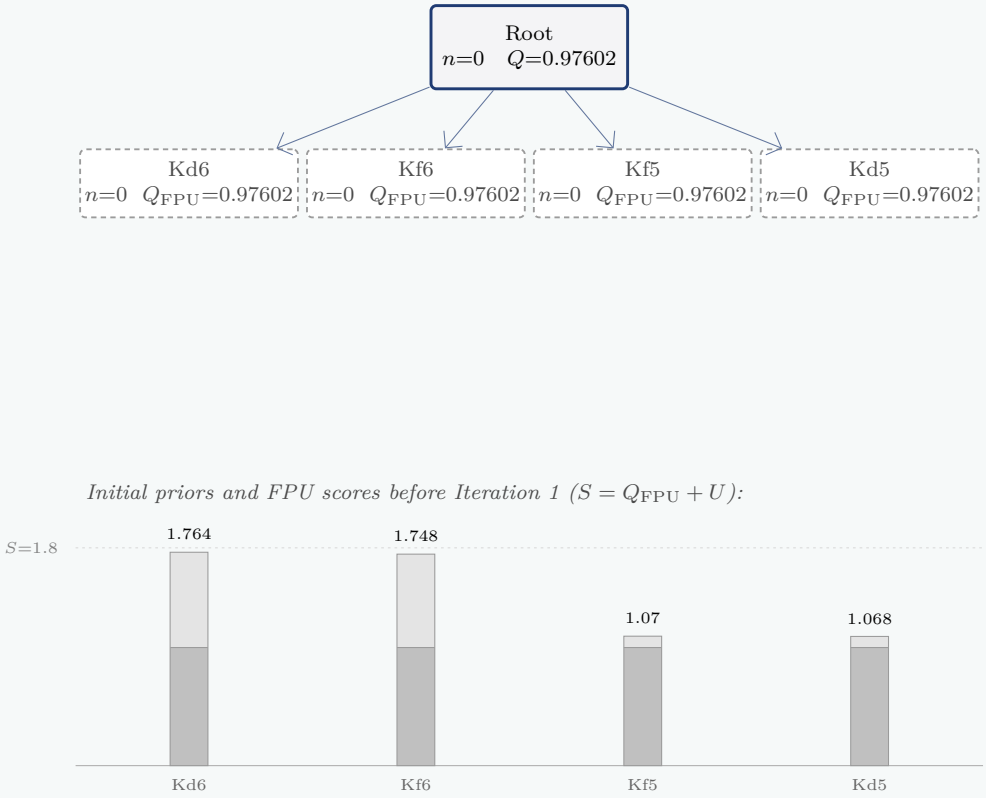


Figure 3.3: FIG-2.0: Iteration 0 (Starting State). All four children unvisited (dashed grey). All Q values are equal to $Q_{\text{FPU}} = +0.97602$ (Equation (2.2)). Ranking is purely decided by policy priors.

Real engine output: Iteration 1

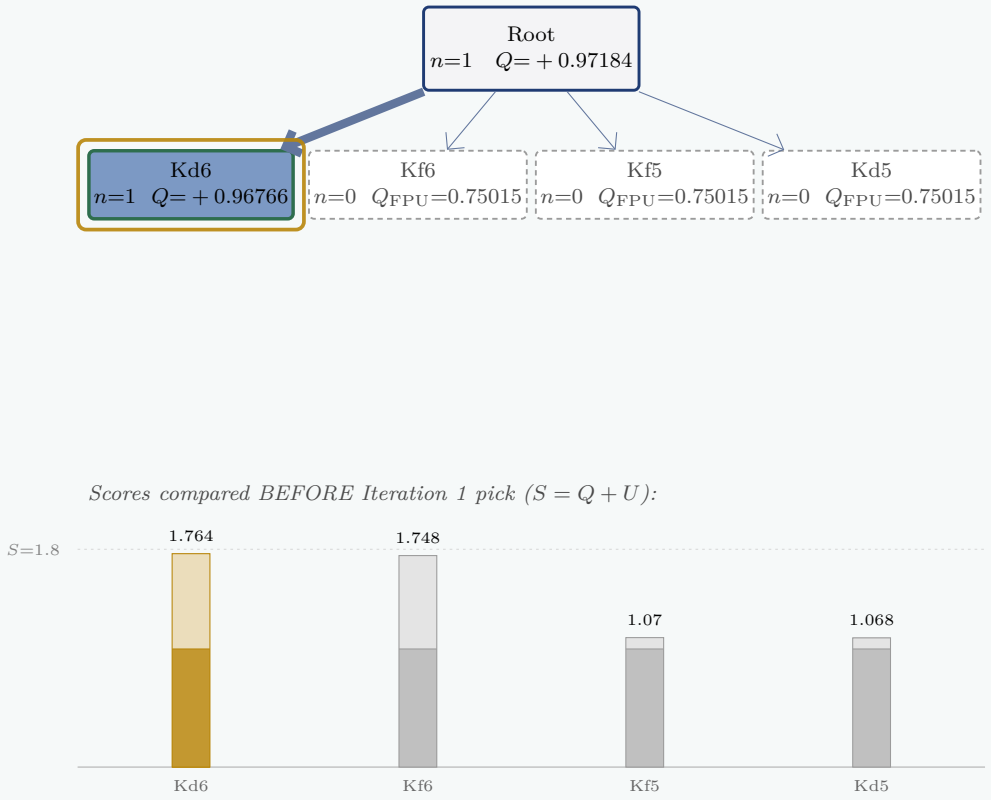


Figure 3.4: FIG-2.1: Iteration 1 (Selection → Expansion → Evaluation → Backpropagation). Kd6 selected ($S = 1.76358$, Equation (2.1)). Expanded leaf returns +0.96766. Kd6 becomes solid green burst; backpropagation updates root to $Q = +0.97184$ (Equation (1.4)). Search stopped at depth 1.

Real engine output: Iteration 2

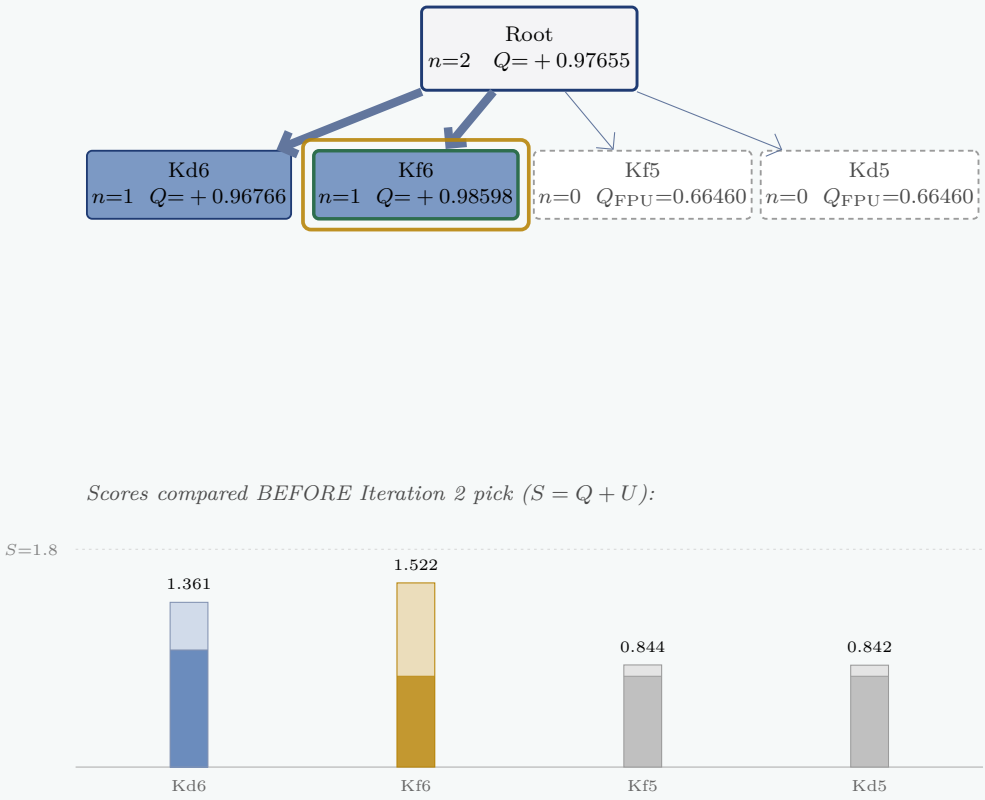


Figure 3.5: FIG-2.2: Iteration 2 (Selection → Expansion → Evaluation → Backpropagation). Kd6 has the best measured score (+0.96766 vs pre-pick FPU 0.75015, grey bars) yet is **not** selected. Its U halved ($1+0 \to 1+1$ denominator in Equation (2.1)), allowing Kf6 ($S = 1.52205$) to take the lead. Post-pick FPU updates to 0.66460 (Equation (2.2)).

Real engine output: Iteration 3

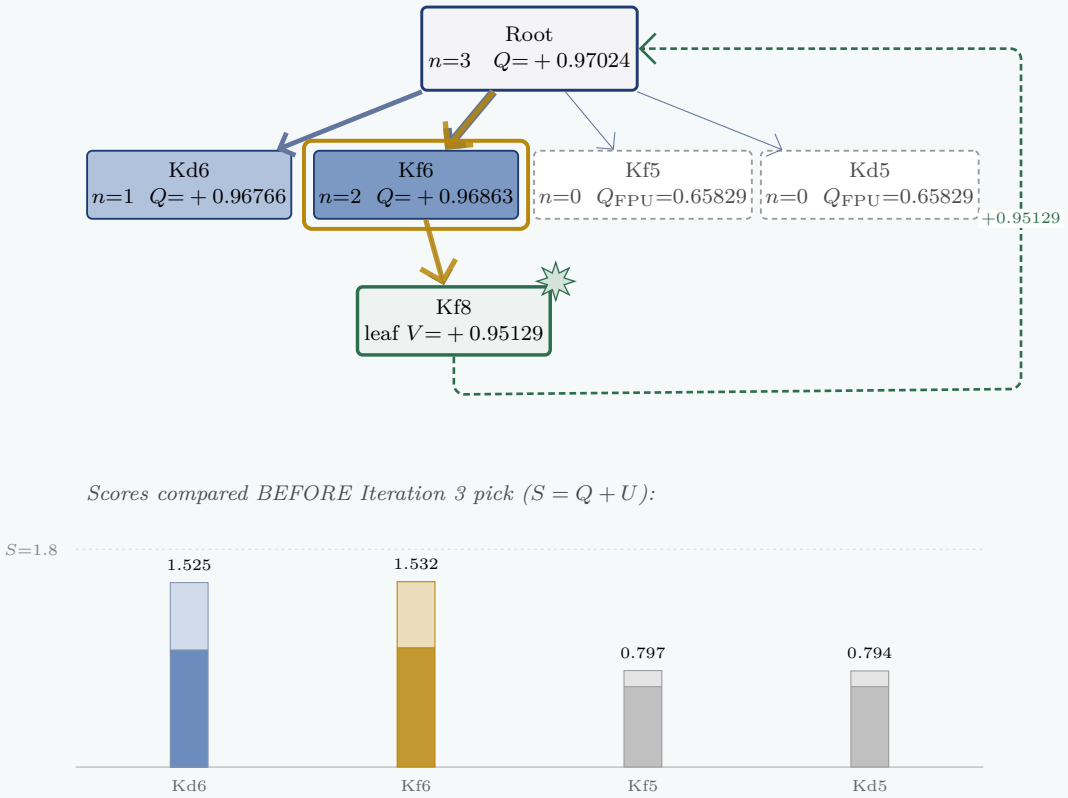


Figure 3.6: FIG-2.3: Iteration 3 (Selection recursed twice before expanding). Kf6 selected ($S = 1.53183$). Selection does not stop at root; it recurses down Kf6 to expand leaf Kf8 (depth 2). Leaf value +0.95129 backs up.

Real engine output: Iteration 4

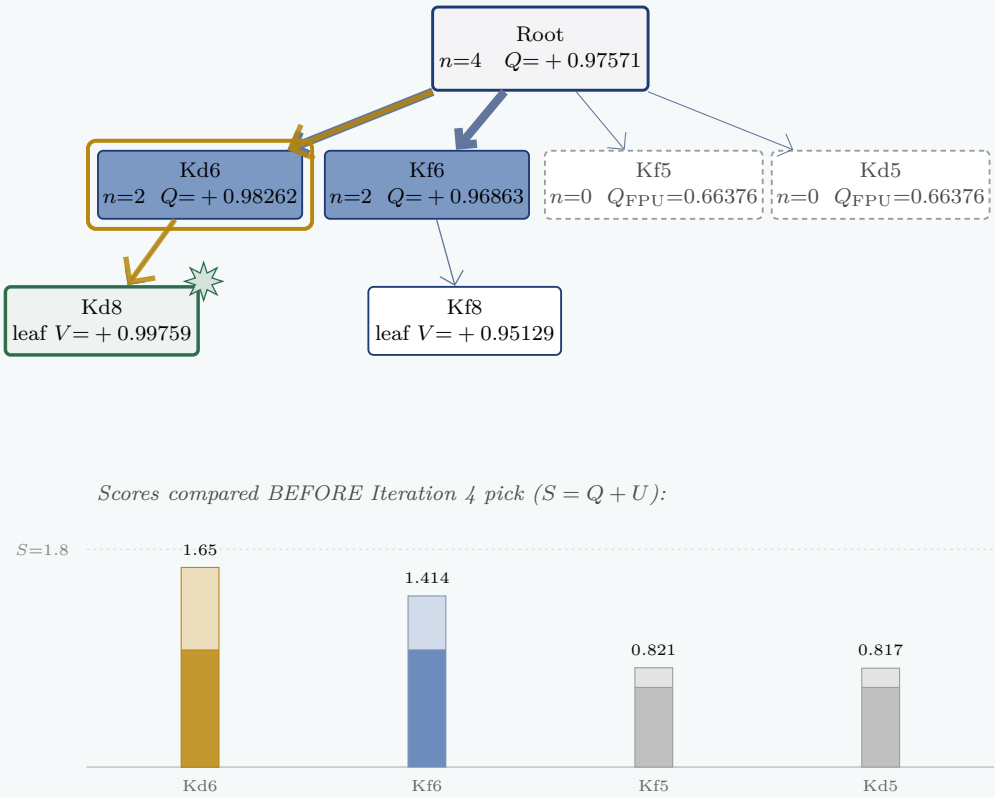


Figure 3.7: FIG-2.4: Iteration 4 (Selection recursed twice before expanding). Kd6 selected ($S = 1.64983$). Selection recurses down Kd6 branch to expand new leaf Kd8 (depth 2). Leaf returns +0.99759, updating Kd6's average Q to +0.98262.

Real engine output: Iteration 5

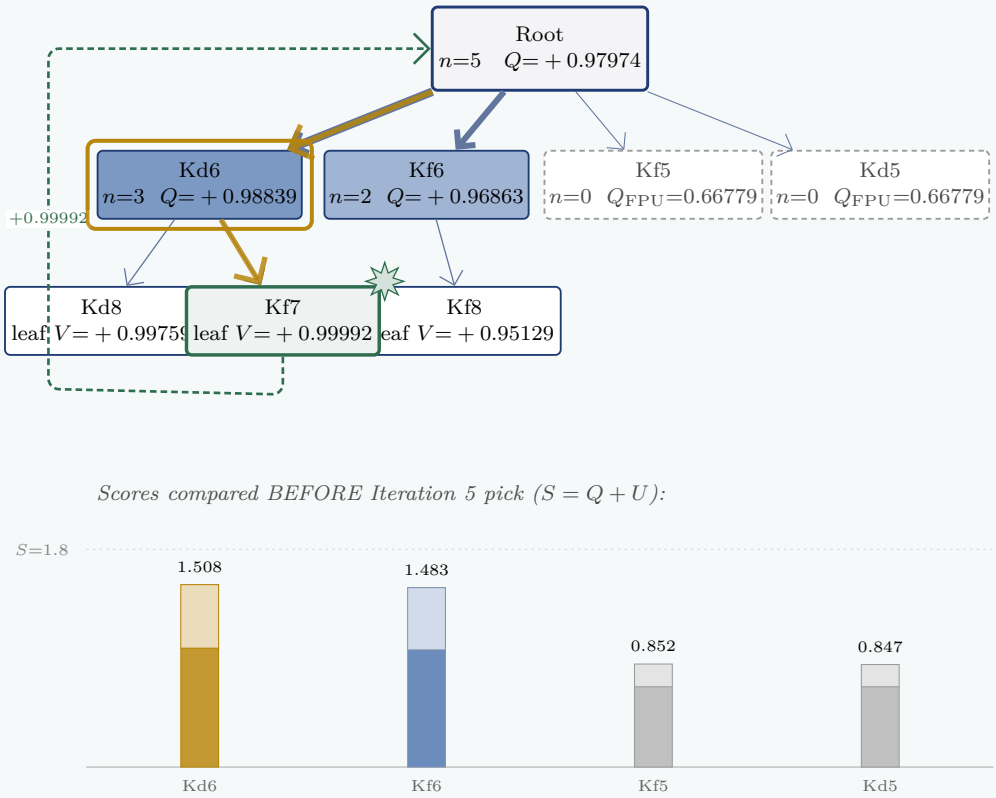


Figure 3.8: FIG-2.5: Iteration 5 (Selection recursed twice before expanding). Kd6 selected ($S = 1.50779$). Selection recurses down Kd6 to expand Kf7 (depth 2). Leaf returns +0.99992. Kd6 subtree widens.

Real engine output: Iteration 6

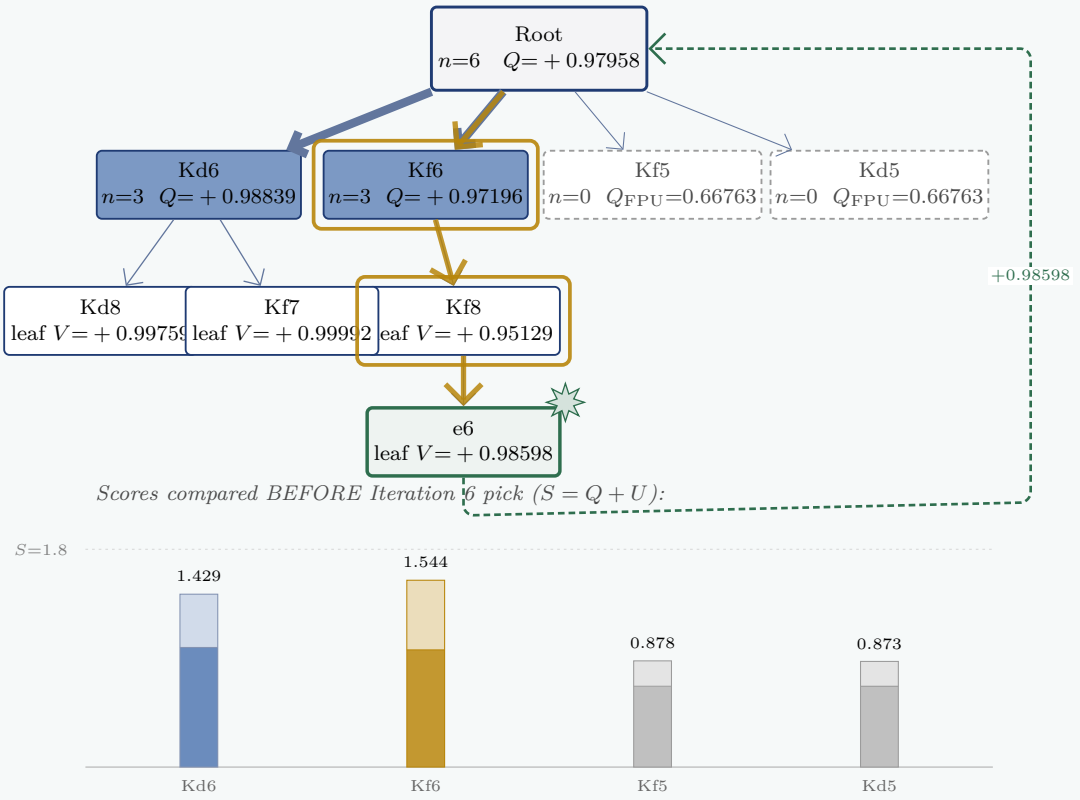


Figure 3.9: FIG-2.6: Iteration 6 (Selection recursed three times before expanding). Kf6 selected ($S = 1.54411$). Selection reaches depth 3 at e6 (under Kf8). Leaf returns +0.97860. Search deepens autonomously.

Real engine output: Iteration 7

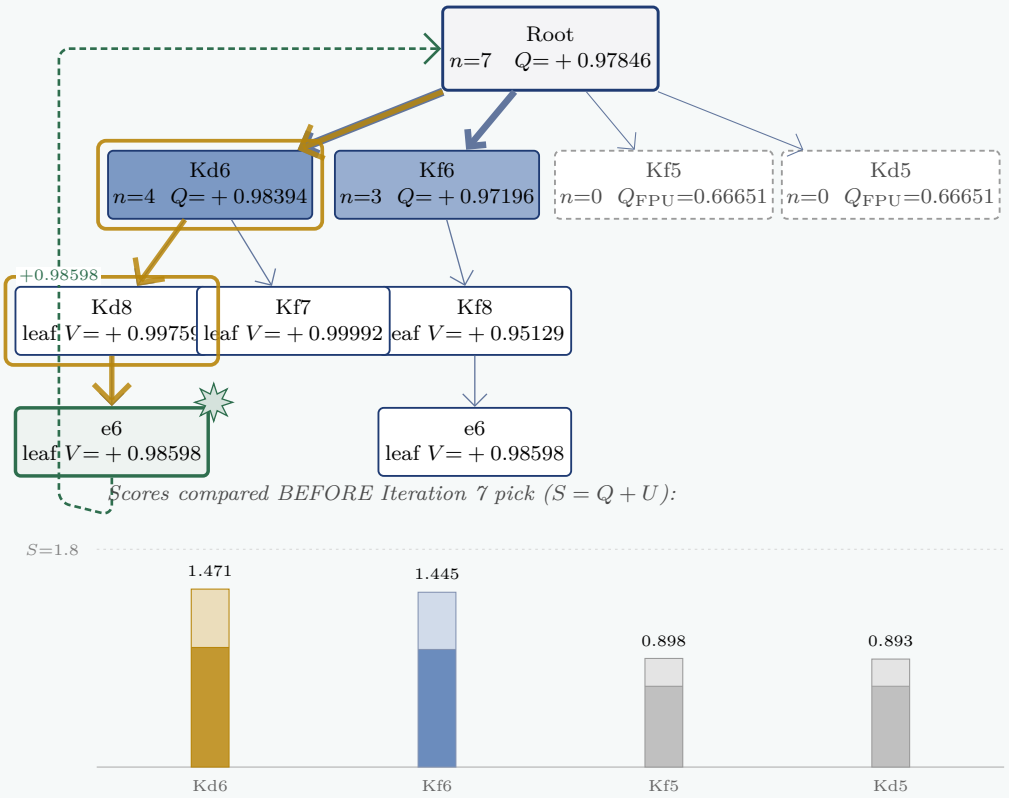


Figure 3.10: FIG-2.7: Iteration 7 (Selection recursed three times before expanding). Kd6 selected ($S = 1.47084$). Selection reaches depth 3 at e6 (under Kd8). Leaf returns +0.97060.

Real engine output: Iteration 8

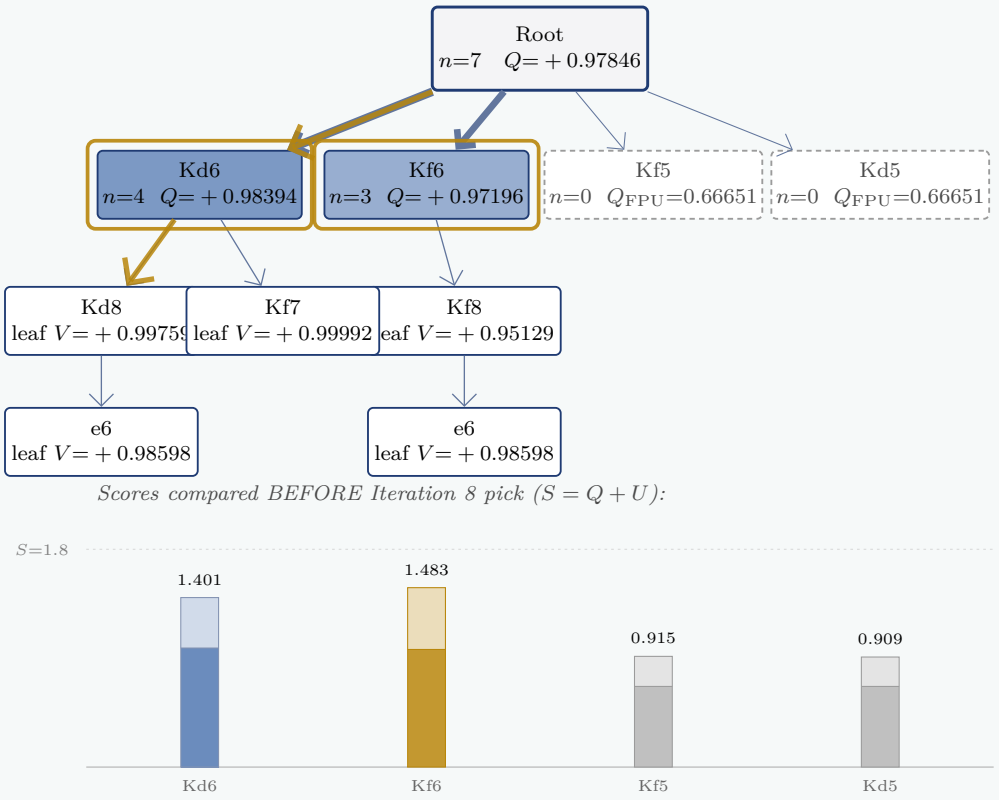


Figure 3.11: FIG-2.8: Iteration 8 (8 full MCTS cycles complete). State after 7 backed-up values. Hand arithmetic matches `lc0.exe go nodes 8` visit counts and Q values to 5 decimal places.

Real engine output: Search Time-Lapse

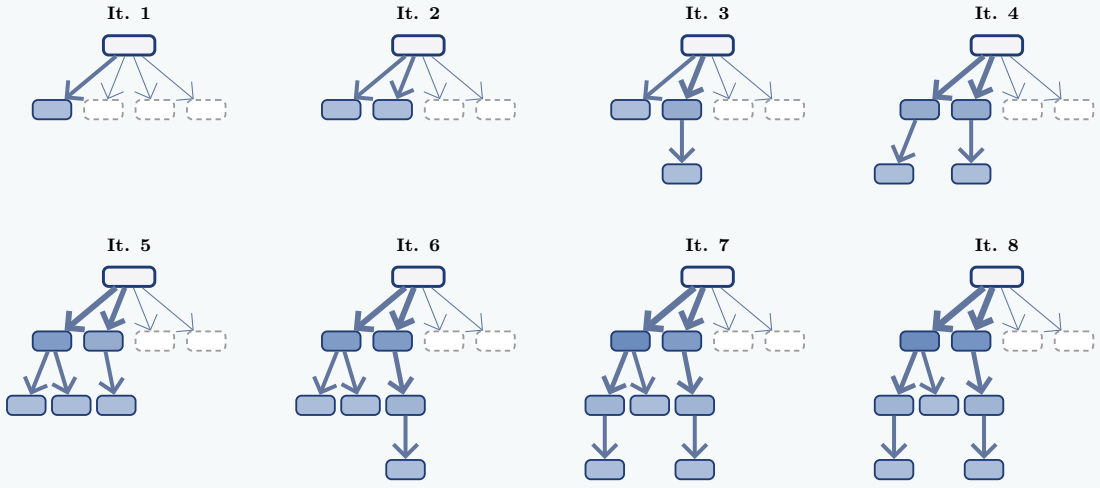
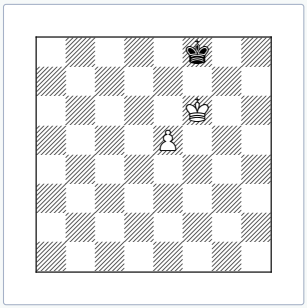


Figure 3.12: FIG-2.9: The whole search time-lapse. Sequence of tree states across iterations 1 to 8 showing structural growth.

3.3 The Sign Flip Convention

Evaluations are recorded from the point of view of the player to move at that node. As values walk back up the tree, the sign flips at every ply (Equation (1.8)):

Real engine output: Sign Flips

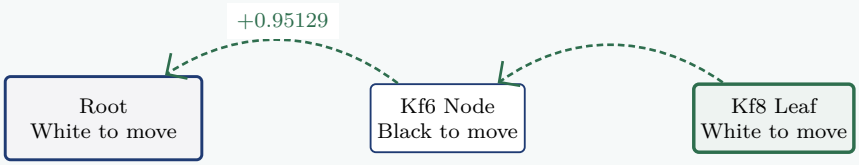


Leaf Node (after 1.Kf6 Kf8)
Side to move: **White**
Evaluated $V = +\mathbf{0.95129}$
"White has a winning advantage (+0.951)."

(a) Leaf Evaluation (White POV)

Incoming Edge to Kf8 (Black to move)
Black evaluates White's +0.95129 win as $-\mathbf{0.95129}$ for Black.

(b) Black-to-move Edge



Root Edge (White to move)
Sign flips back: White receives $+\mathbf{0.95129}$.
Kf6 running average: $(0.98598 + 0.95129)/2 = \mathbf{0.968635}$.

(c) Root Edge (White POV)

Figure 3.14: FIG-2.10: Iteration 3 Sign Flips. New leaf after 1.Kf6 Kf8 gives +0.95129 (White). Black edge receives −0.95129 via Negamax sign flip (Equation (1.8)). Root edge receives +0.95129, updating Kf6 average to 0.968635.

3.4 Emergent Depth

Established mechanism: Autonomous Search Expansion

Depth Emergence across Iterations

Iterations 1–2: Stop at root's children (**depth 1**)
Iterations 3–5: Reach **depth 2** (Kf8, Kd8, Kf7 appear)
Iterations 6–7: Reach **depth 3** (e6 pawn moves appear)

Leela has no depth parameter at all. Depth is what happens when U collapses.

Figure 3.15: FIG-2.11: Depth Emergence. Iterations 1–2 stop at depth 1; 3–5 reach depth 2; 6–7 reach depth 3. Depth is an emergent outcome of U decay (Equation (2.1)), not a pre-configured parameter.

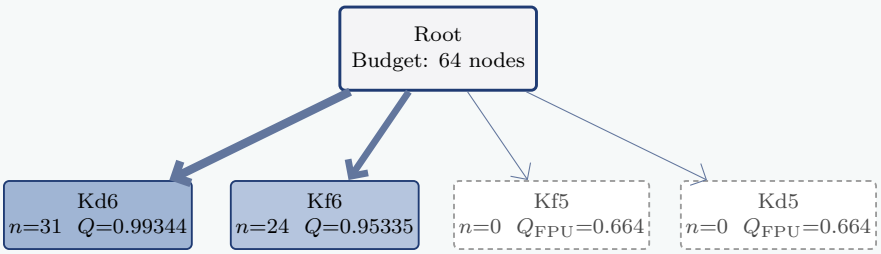
Chapter 4

Why the Search Abandons a Move

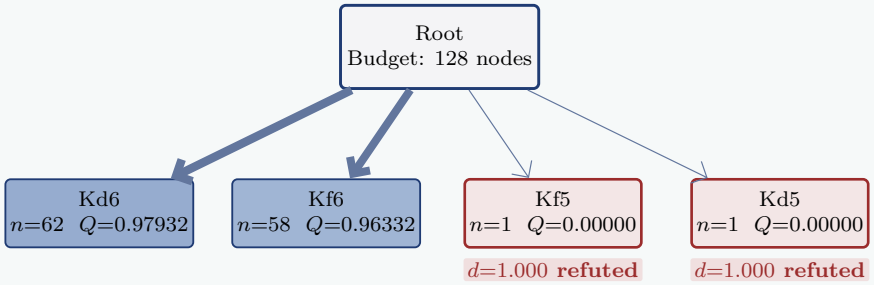
We examine how the search handles bad candidate moves across expanded visit budgets.

4.1 The Refutation Ladder

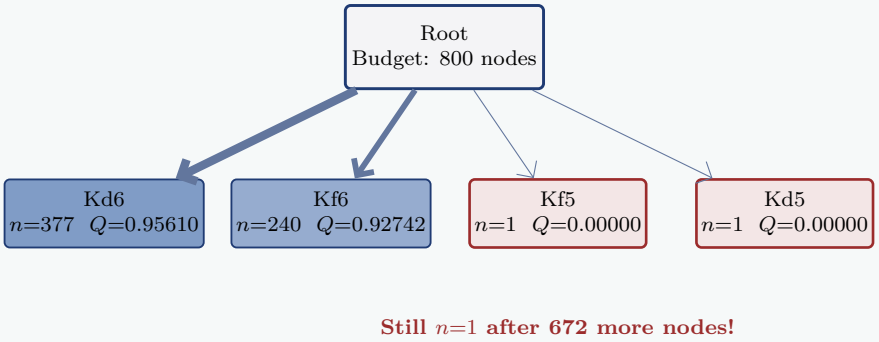
Real engine output: Refutation Ladder



(a) 64 nodes: Kd6 N=31, Kf6 N=24



(b) 128 nodes: Kf5 & Kd5 visited once ($Q = 0.000$)

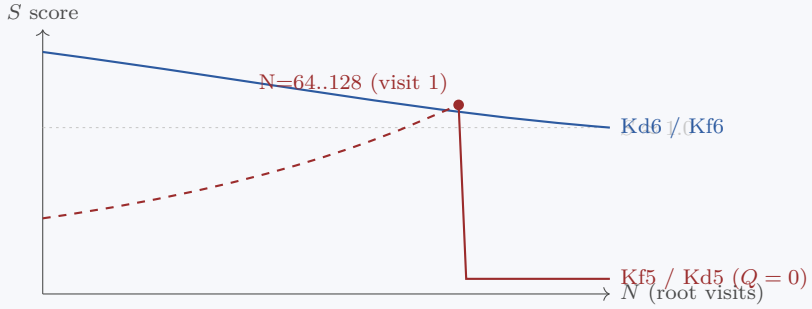


(c) 800 nodes: Kd6 N=377, Kf6 N=240, drawing moves stay at N=1

Figure 4.2: FIG-3.1: The refutation ladder. Between 64 and 128 nodes, Kf5 and Kd5 each receive exactly one visit, return $Q = 0.000$ (Equation (1.4)), and are permanently abandoned for the next 672 nodes.

4.2 The Score Decay Race

Established mechanism: PUCT Score Curves

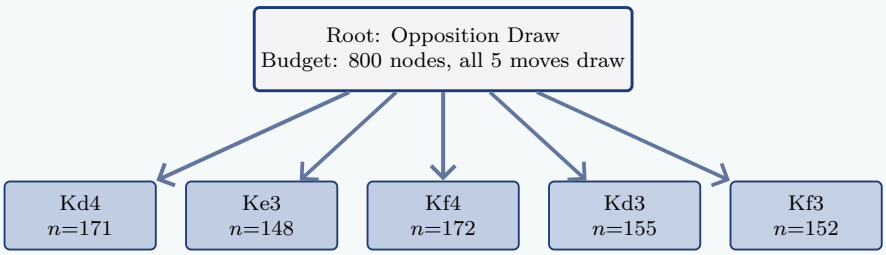


Curves derived via PUCT formula from measured priors; markers at measured $N = 64, 128$.

Figure 4.3: FIG-3.2: PUCT score race. Solid markers represent measured engine budgets ($N = 64, 128$). Dashed curves represent theoretical $S(N) = Q_{\text{FPU}}(N) + U(N)$ derived via the PUCT formula (Equations (2.1)–(2.2)) as unvisited U grows with $\sqrt{N}$ until a single visit drops Q to zero.

4.3 Contrast: When Nothing Matters

Real engine output: Dead Drawn Positions



Flat visit distribution across all legal moves: position is drawn.

Figure 4.4: FIG-3.3: Opposition Draw Contrast. In a dead drawn position, visit counts split almost evenly across all 5 legal moves ($N = 171, 148, 172, 155, 152$). A flat tree indicates no tactical differentiation.

4.4 The Honest Caveat

Established mechanism: Single-Look vs Deep Refutation

Refutation in 1 Look
Value head catches draw/loss immediately on first visit ($Q = 0.000$). Search abandons move.

Refutation 5 Plies Deep
Value head initial look is optimistic. Search must build subtree before truth emerges.

Figure 4.5: FIG-3.4: One-look vs deep refutation. Rapid refutation relies on the value head recognizing drawn/lost structures on the first visit (Equation (1.2)). Deep tactical refutations require building multi-ply subtrees.

Chapter 5

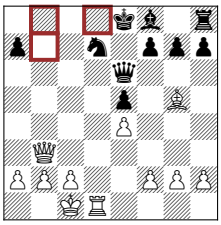
Reading What the Engine Decided

We analyze how visit counts and evaluation proofs determine the final played move.

5.1 The Moment the Engine Changed Its Mind

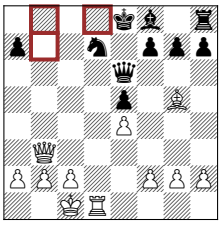
In the Morphy position (`4kb1r/p2n1ppp/4q3/4p1B1/4P3/1Q6/PPP2PPP/2KR4 w k - 0 16`), white has a forced mate in 2 starting with 16.Qb8+!

Real engine output: Morphy Mind Change



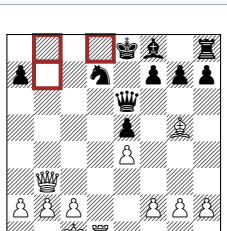
Morphy (64 nodes)
Qb7: $n = 26$, $Q = 0.56874$
Qb8+: $n = 0$ (unvisited)
Engine plays: **Qb7**

(a) 64 nodes: Qb7 (bestmove)



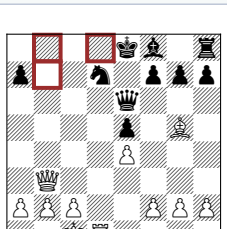
Morphy (128 nodes)
Qb7: $n = 53$, $Q = 0.62435$
Qb8+: $n = 0$ (unvisited)
Engine plays: **Qb7**

(b) 128 nodes: Qb7 (bestmove)



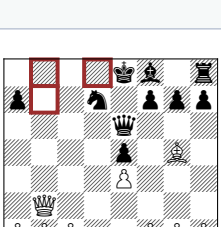
Morphy (400 nodes)
Qb7: $n = 169$, $Q = 0.65752$
Qb8+: $n = 0$ (unvisited)
Engine plays: **Qb7**

(c) 400 nodes: Qb7 (bestmove)



Morphy (1600 nodes)
Qb7: $n = 172$, $Q = 0.65874$
Qb8+: $n = 3$, $Q = 1.00000$ **[PROOF]**
Engine plays: **Qb8+**

(d) 1600 nodes: Qb8+ (N=3, Q=1.000)



Morphy (6400 nodes)
Qb7: $n = 172$, $Q = 0.65874$
Qb8+: $n = 3$, $Q = 1.00000$ **[PROOF]**
Engine plays: **Qb8+**

(e) 6400 nodes: Qb8+ (identical state)

Figure 5.2: FIG-4.1: Mind change progression. Through 400 nodes, Qb8+ has zero visits. At 1600 nodes, 3 visits yield $Q = 1.000$ (proof of mate), causing bestmove to flip from Qb7 to Qb8+ via PUCT selection (Equation (2.1)). At 6400 nodes, state remains identical.

5.2 Proof Beats Sampling

Real engine output: Exact Proofs vs Statistical Sampling

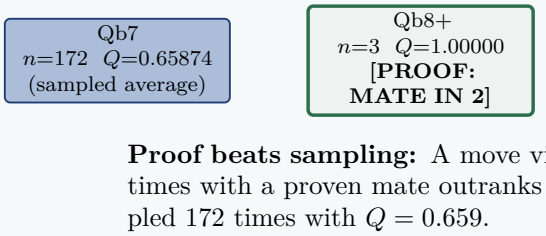


Figure 5.3: FIG-4.2: Proof outranks sampling. A move visited 3 times with a proven mate ($Q = 1.000$) outranks a move sampled 172 times with $Q = 0.659$ (Equation (1.4)).

5.3 Policy Proposes, Value Disposes

Real engine output: Policy and Value Interaction

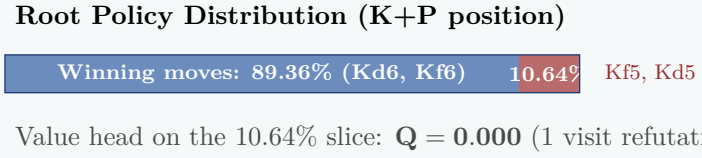


Figure 5.4: FIG-4.3: Policy proposes, value disposes. Policy assigns 89.36% to winning moves and 10.64% to draws. Value head scores the drawing slice 0.000 on first visit (Equation (1.2)).

5.4 Which Number Chooses the Move?

Established mechanism: Visit Ranking vs Evaluation

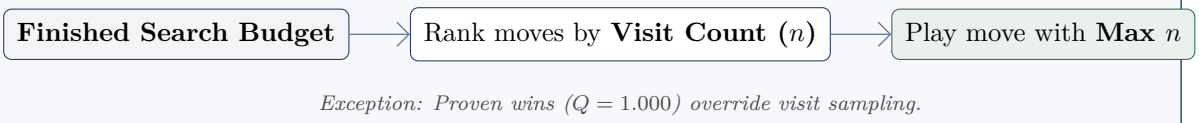


Figure 5.5: FIG-4.4: Move decision flowchart. Candidates are ranked by visit count n_a. High Q on low n_a is a question, not an answer, unless $Q = 1.000$ (proof).

Chapter 6

Inside the Network

This chapter details internal network processing, clearly separating established architecture and measured outputs from unverified hypotheses.

6.1 The Conveyor Belt

Established mechanism: Illustrative Role Overlay

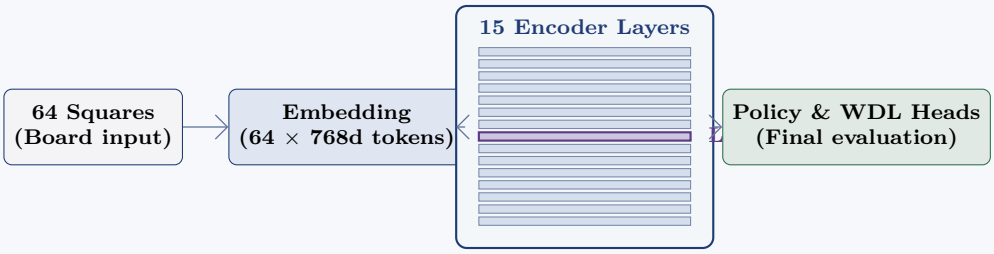
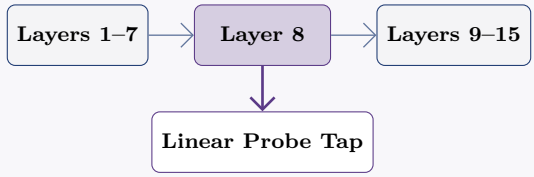


Figure 6.1: FIG-5.1: Conveyor belt architecture. 64 square tokens pass through 15 transformer encoder layers (24 heads each) to policy prior $P(a \mid s)$ and WDL value heads (Equations (1.1)–(1.2)). Layer role progression (pieces → geometry → lookahead → strategy) is an unmeasured illustrative cartoon.

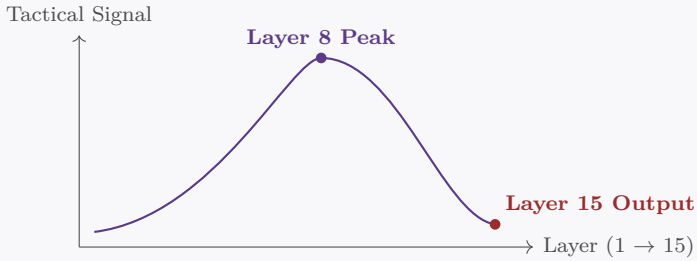
6.2 The Linear Probe Hypothesis

This project's hypothesis — not yet verified: Linear Probe Model



Tapping middle-layer representations.

(a) Forward Pass Probe Tap



Conceptual shape of suppressed-win hypothesis (no numbers).

(b) Suppressed Sacrifice Hypothesis

Figure 6.3: FIG-5.2: Probe hypothesis. Conceptual model of middle-layer tactical awareness suppressed by final policy output. No fabricated internal probe values are used.

6.3 Measured Counterpart: Policy Suppression

Real engine output: Policy Suppression of Forced Mate

Real Engine Output: Morphy Position Policy Pass

$P(\text{Qb8+}) = \mathbf{1.60\%}$ *(Network instinct barely considered the mate)*

$Q(\text{Qb8+}) = \mathbf{1.00000}$ *(Search found mate at $n = 3$ and overrode policy)*

Measured proof of policy override without needing an internal probe hypothesis.

Figure 6.4: FIG-5.3: Measured instinct override. In the Morphy position, $P(\text{Qb8+}) = 1.60\%$. The instinct nearly missed a forced mate in 2, providing a measured counterpart to policy suppression without probe assumptions.

6.4 Attention Map Visualization Tooling

This project's hypothesis — not yet verified: Tooling Limitation & Target Concept

Current Visual Tooling
Attention averaged over all 15 layers × 24 heads (diffuse map).

Single Head Target
Single head attention map (future goal — drawn hollow).

Figure 6.5: FIG-5.4: Attention maps. Current visualizations average attention across all 15 layers and 24 heads (diffuse map). Single-head isolation remains a target concept.

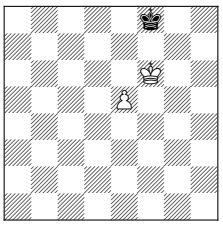
Chapter 7

Traps for System Developers

We highlight critical technical gotchas when building software tools on top of LC0 evaluations.

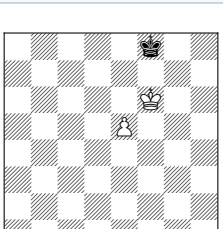
7.1 The FEN-vs-History Discrepancy

Real engine output: Bare FEN vs FEN with History



Bare FEN Evaluation
`position fen 5k2/8/... w - - 2 2`
$V = +\mathbf{0.98838}$, $d = \mathbf{0.012}$
Fabricated history repeats position.

(a) Bare FEN: $V = +0.98838, d = 0.012$



FEN-with-History Evaluation
`position fen 4k3/... moves e6f6 e8f8`
$V = +\mathbf{0.95129}$, $d = \mathbf{0.049}$
True game context used during search.

(b) FEN-with-History: $V = +0.95129, d = 0.049$

Figure 7.2: FIG-6.1: Same board, two histories. Evaluating the position after 1.Kf6 Kf8 with bare FEN vs move history yields a 0.037 shift in net value V (Equation (1.2)) and 4× draw probability, because bare FENs force the engine to fabricate repeated position history.

7.2 Closing Summary Card

Product Visual Guide Summary

- **Policy Arrows** → Prior attention (FIG-1.3)
- **Variations** → Visit-weighted tree paths (FIG-2.9)
- **Refuted Moves** → One-visit $Q = 0$ drops (FIG-3.1)
- **High Q, Low n** → Questions, not answers (FIG-4.4)

Figure 7.3: FIG-6.2: Product visual guide mapping summary card. Reference mapping UI display features to underlying search engine mechanics (Equations (1.1)–(2.3)).